

Algoritmos de Enrutamiento

Repositorio: <https://github.com/paulabaal12/LAB3-REDES>

Video de demostración: <https://youtu.be/g-RyAg4sJpc>

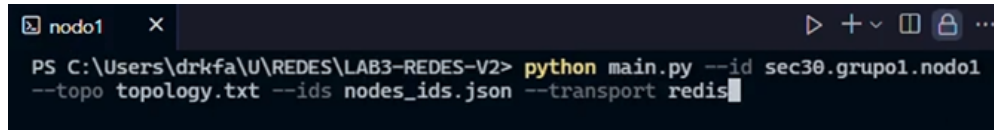
Arquitectura del Laboratorio

El sistema implementa un simulador distribuido de protocolos de enrutamiento con las siguientes partes principales:

- [main.py](#)
 - Inicializa la topología a partir de archivos de configuración (topology.txt y nodes_ids.json).
 - Crea el transporte de mensajes mediante Redis utilizando LSR + Flooding
 - Ejecuta tareas concurrentes en asyncio:
 - Envío periódico de mensajes hello a los vecinos.
 - Difusión (flooding) de información de enlaces descubiertos.
 - Descuento del tiempo de vida (TTL) de las conexiones.
 - Menú interactivo para mostrar la topología o calcular rutas con Dijkstra.
- [topology.py](#)
 - Convierte los identificadores cortos de nodos en IDs completos.
 - Carga los enlaces iniciales de la red y los transforma en una estructura de diccionarios con pesos y temporizadores.
 - Permite consultar los vecinos directos de cada nodo.
- [transport.py](#)
 - Define la clase AsyncRedisTransport, que abstrae el envío y recepción de mensajes entre nodos usando Redis Pub/Sub.
 - Cada nodo escucha únicamente en su propio canal (su ID completo).
 - Los mensajes se publican en el canal del nodo destino.
- routing/[dijkstra.py](#)
 - Implementa el algoritmo de Dijkstra para calcular las rutas más cortas en la topología aprendida.
 - Construye una tabla de ruteo con destino, siguiente salto y costo acumulado.
 - Permite exportar resultados en un archivo de texto (output/single_table.txt) y en formato CSV (output/table.csv).

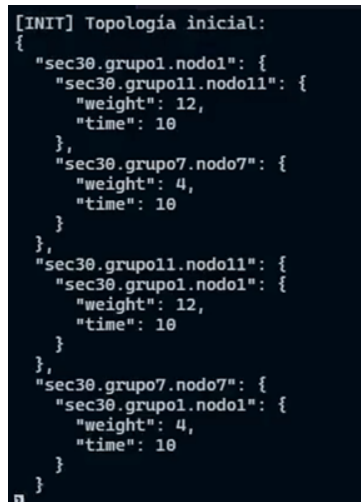
Ejecución

- Para iniciar un nodo:



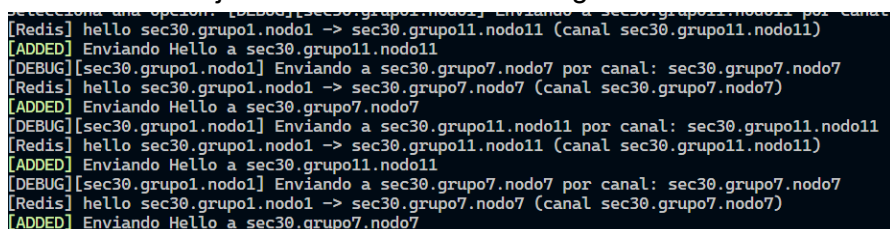
```
PS C:\Users\drkfa\U\REDES\LAB3-REDES-V2> python main.py --id sec30.grupo1.nodo1
--topo topology.txt --ids nodes_ids.json --transport redis
```

- En consola se muestra:
 - El estado inicial de la topología cargada.



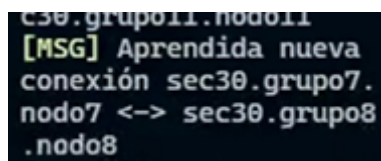
```
[INIT] Topologia inicial:
{
  "sec30.grupo1.nodo1": {
    "sec30.grupo11.nodo11": {
      "weight": 12,
      "time": 10
    },
    "sec30.grupo7.nodo7": {
      "weight": 4,
      "time": 10
    }
  },
  "sec30.grupo11.nodo11": {
    "sec30.grupo1.nodo1": {
      "weight": 12,
      "time": 10
    }
  },
  "sec30.grupo7.nodo7": {
    "sec30.grupo1.nodo1": {
      "weight": 4,
      "time": 10
    }
  }
}
```

- Envío de mensajes hello a los vecinos configurados.



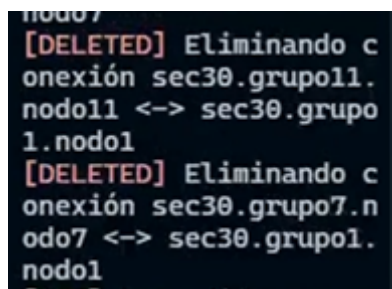
```
[Redis] hello sec30.grupo1.nodo1 -> sec30.grupo11.nodo11 (canal sec30.grupo11.nodo11)
[ADDED] Enviando Hello a sec30.grupo11.nodo11
[DEBUG][sec30.grupo1.nodo1] Enviando a sec30.grupo7.nodo7 por canal: sec30.grupo7.nodo7
[Redis] hello sec30.grupo1.nodo1 -> sec30.grupo7.nodo7 (canal sec30.grupo7.nodo7)
[ADDED] Enviando Hello a sec30.grupo7.nodo7
[DEBUG][sec30.grupo1.nodo1] Enviando a sec30.grupo11.nodo11 por canal: sec30.grupo11.nodo11
[Redis] hello sec30.grupo1.nodo1 -> sec30.grupo11.nodo11 (canal sec30.grupo11.nodo11)
[ADDED] Enviando Hello a sec30.grupo11.nodo11
[DEBUG][sec30.grupo1.nodo1] Enviando a sec30.grupo7.nodo7 por canal: sec30.grupo7.nodo7
[Redis] hello sec30.grupo1.nodo1 -> sec30.grupo7.nodo7 (canal sec30.grupo7.nodo7)
[ADDED] Enviando Hello a sec30.grupo7.nodo7
```

- Descubrimiento de nuevas conexiones mediante flooding.



```
[MSG] Aprendida nueva
conexión sec30.grupo7.
nodo7 <-> sec30.grupo8
.nodo8
```

- Eliminación de enlaces inactivos.



```
[DELETED] Eliminando c
onexión sec30.grupo11.
nodo11 <-> sec30.grupo
1.nodo1
[DELETED] Eliminando c
onexión sec30.grupo7.n
odo7 <-> sec30.grupo1.
nodo1
```

- Al ingresar un 2 en el Menú, se mostrará lo siguiente:
 - En el archivo `single_table.txt` se escribirá la tabla del nodo seleccionado:



```

output > single_table.txt
1
2  Tabla de ruteo para nodo sec30.grupo9.nodo9:
3
4  Destino                Next Hop                Costo
5  -----
6  sec30.grupo1.nodo1     sec30.grupo5.nodo5     25
7  sec30.grupo10.nodo10   sec30.grupo4.nodo4     22
8  sec30.grupo11.nodo11   sec30.grupo4.nodo4     32
9  sec30.grupo2.nodo2     sec30.grupo4.nodo4     25
10 sec30.grupo3.nodo3      sec30.grupo5.nodo5     24
11 sec30.grupo4.nodo4      sec30.grupo4.nodo4     8
12 sec30.grupo5.nodo5      sec30.grupo5.nodo5     10
13 sec30.grupo6.nodo6      sec30.grupo4.nodo4     37
14 sec30.grupo7.nodo7      sec30.grupo5.nodo5     21
15 sec30.grupo8.nodo8      sec30.grupo4.nodo4     16
16
17

```

- Así mismo, en `table.csv` se irá rellenando la tabla con los costos calculados, para que entonces al ejecutar `dijkstra` en todos los nodos, la tabla se llene con todos los costos de la topología:



```

output > table.csv > data
1 source,sec30.grupo1.nodo1,sec30.grupo2.nodo2,sec30.grupo3.nodo3,se
2 sec30.grupo9.nodo9,25,25,24,8,10,37,21,16,,22,32
3

```

Ejecución y Resultados

1. Se levantan los 11 nodos:

The screenshot shows a terminal window with multiple tabs. The active tab displays a series of commands and their outputs for setting up 11 nodes. The commands include creating interfaces, assigning IP addresses, and starting network services. The outputs show the successful execution of these commands and the resulting network configuration for each node.

2. Se calcula Dijkstra del nodo 9 y se compara con la tabla real:

source	N1	N2	N3	N4	N5	N6	N7	N8	N9	N10	N11
N1	0	21	26	19	15	17	4	11	25	7	12
N2	21	0	10	17	24	29	17	20	25	14	28
N3	26	10	0	23	14	36	22	15	24	21	31
N4	19	17	23	0	18	29	15	8	8	14	24
N5	15	24	14	18	0	27	11	18	10	14	22
N6	17	29	36	29	27	0	16	21	37	15	5
N7	4	17	22	15	11	16	0	7	21	3	11
N8	11	20	15	8	18	21	7	0	16	6	16
N9	25	25	24	8	10	37	21	16	0	22	32
N10	7	14	21	14	14	15	3	6	22	0	14
N11	12	28	31	24	22	5	11	16	32	14	0

2. Tabla de ruteo para nodo sec30.grupo9.nodo9:

Destino	Next Hop	Costo
sec30.grupo1.nodo1	sec30.grupo5.nodo5	25
sec30.grupo10.nodo10	sec30.grupo4.nodo4	22
sec30.grupo11.nodo11	sec30.grupo4.nodo4	32
sec30.grupo2.nodo2	sec30.grupo4.nodo4	25
sec30.grupo3.nodo3	sec30.grupo5.nodo5	24
sec30.grupo4.nodo4	sec30.grupo4.nodo4	8
sec30.grupo5.nodo5	sec30.grupo5.nodo5	10
sec30.grupo6.nodo6	sec30.grupo4.nodo4	37
sec30.grupo7.nodo7	sec30.grupo5.nodo5	21
sec30.grupo8.nodo8	sec30.grupo4.nodo4	16

3. Al calcular Dijkstra con todos los nodos, la tabla general se llena:

source	N1	N2	N3	N4	N5	N6	N7	N8	N9	N10	N11
N1	0	21	26	19	15	17	4	11	25	7	12
N2	21	0	10	17	24	29	17	20	25	14	28
N3	26	10	0	23	14	36	22	15	24	21	31
N4	19	17	23	0	18	29	15	8	8	14	24
N5	15	24	14	18	0	27	11	18	10	14	22
N6	17	29	36	29	27	0	16	21	37	15	5
N7	4	17	22	15	11	16	0	7	21	3	11
N8	11	20	15	8	18	21	7	0	16	6	16
N9	25	25	24	8	10	37	21	16	0	22	32
N10	7	14	21	14	14	15	3	6	22	0	14
N11	12	28	31	24	22	5	11	16	32	14	0

output > table.csv > data

```
1 source,sec30.grupo1.nodo1,sec30.grupo2.nodo2,sec30.grupo3.nodo3,sec30.grupo4.nodo4,sec30.grupo5.nodo5,sec30.grupo6.nodo6,sec30.grupo7.nodo7,sec30.grupo8.nodo8,sec30.grupo9.nodo9,sec30.grupo10.nodo10,sec30.grupo11.nodo11
2 sec30.grupo1.nodo1,21,26,19,15,17,4,11,25,7,12
3 sec30.grupo2.nodo2,21,10,17,24,29,17,20,25,14,28
4 sec30.grupo3.nodo3,26,10,23,14,36,22,15,24,21,31
5 sec30.grupo4.nodo4,19,17,23,0,18,29,15,8,8,14,24
6 sec30.grupo5.nodo5,15,24,14,18,0,27,11,18,10,14,22
7 sec30.grupo6.nodo6,17,29,36,29,27,0,16,21,37,15,5
8 sec30.grupo7.nodo7,4,17,22,15,11,16,0,7,21,3,11
9 sec30.grupo8.nodo8,11,20,15,8,18,21,7,0,16,6,16
10 sec30.grupo9.nodo9,25,25,24,8,10,37,21,16,0,22,32
11 sec30.grupo10.nodo10,7,14,21,14,14,15,3,6,22,0,14
12 sec30.grupo11.nodo11,12,28,31,24,22,5,11,16,32,14,0
```